Sprint 1 — Design Document

Project 01B: Finding the Next Best Step

Team DSIders

Sprint 0 left us with two findings. Our users are not short of support; they are short of a way to navigate it. And they often cannot tell which piece of missing information is the one holding them up. This document turns those findings into something we can actually build: a tool that starts from what a person wants to become, asks a few questions, and tells them one thing to do next.

The scope narrowed during this sprint. We are not building a catalogue of schemes with an apply button. The product answers doubts about a career the user already has in mind.

The clickable prototype is on Figma:

`https://www.figma.com/design/9K2MGda8rKSc7ZYwwkkB81/Untitled?node-id=0-1&p=f&t=PYJnIZwnkmY4PRez-0`

1 User context and pain-point analysis

The user is a final-year student at a government or Tier-2 college in Telangana, enrolled in a Barabari Collective programme. They have usually decided what they want to do. What they lack is the route from that decision to a specific action.

The gap shows up between Barabari's scheduled touchpoints. A student finishes a training module, runs into a question on a Tuesday evening, and there is no mentor in the room. The programme is well run. It simply cannot be present for every decision a student faces.

We did consider building for the caseworker instead, since they answer the same questions all day. But a caseworker knows what they do not know, and a student often does not. The student's version of the problem is harder, so we went with the student.

Four pain points came out of our Sprint 0 work:

1. **No translation layer.** Students can state a goal. They cannot break it down into a first step.
2. **Unknown unknowns.** If you know you are missing something, you can look it up. A student who has never been told that courses and interviews expect a resume has nothing to search for. This is the pain point we are mainly aiming at.
3. **A missing item reads as a rejection.** Portals and listings treat a blank field as disqualification. In practice a missing resume is two hours of work, not a verdict.
4. **Overload.** Long forms and long lists of maybe-relevant options stall people, particularly on a shared phone over a slow connection.

The environment makes all four worse. The interface has to work on low RAM and intermittent 3G, drafts and documents are usually on paper or nowhere, and the vocabulary a course listing uses is not the vocabulary the student has. Someone who cannot name the field they want also cannot search for the course that teaches it.

2 Existing-solution analysis

Several things already address parts of this problem. Each of them works for someone, and each stops short at the same place.

Solution	Works well for	Where it stops for our user
College career cell	The designated route, in theory	Mostly absent; few of these colleges have any guidance staff
myScheme and similar portals	Authoritative, free, wide coverage	Needs administrative vocabulary and a long form first, and a blank field reads as disqualification
General chatbots	Natural phrasing, no form	Invents specifics and has no rule check behind it; a confident wrong answer costs the student a wasted trip
YouTube roadmaps	Free and motivating	Made for a generic viewer, so it cannot say which step this student is on
Resume builders	Fine once you know you need one	Assume the diagnosis has already happened
NGO casework	Accurate and adapts to the person	Limited by geography and counsellor time; not available at the moment the doubt appears

Table 1: Existing options and the point at which each one stops helping.

The last column repeats itself. Portals tell you which programmes match a finished profile, chatbots tell you what your question sounds like, roadmaps describe a generic journey, and a caseworker tells you what to do when they are there. None of them tells a particular student what is missing from their particular picture and what to do about it this week. That is the gap we are working in, and it is a smaller problem than building a better portal.

3 Proposed solution and user flow

We are building a next-step engine. The user says what they want to become, or hands over their resume. The system asks a small number of targeted questions, works out where the person stands, and returns one concrete action. When it is not confident enough, it says so and offers a caseworker rather than guessing.

Four decisions shape the design:

1. **One question at a time.** Instead of a form, the system scores the attributes it could ask about by how much each would narrow the picture, and asks only the best one. Each answer changes the scores. This works out to three or four taps. The interface also states why a question was chosen, so the sequence does not feel arbitrary.

2. **Grouped results with reasons.** Findings come back as *already have*, *missing* and *not urgent yet*, each with a short reason. A ranked list asks the user to trust the order. A reason lets them check it and tell us we are wrong.

3. **A gap is a task.** When something is missing, the system looks for the nearest thing the user could actually do about it and shows that instead of reporting a shortfall.

4. **Low confidence hands over.** The engine tracks confidence through the session. Below a threshold, or when the user says they do not know, it offers a named caseworker and shows what would be passed on.

End-to-end flow

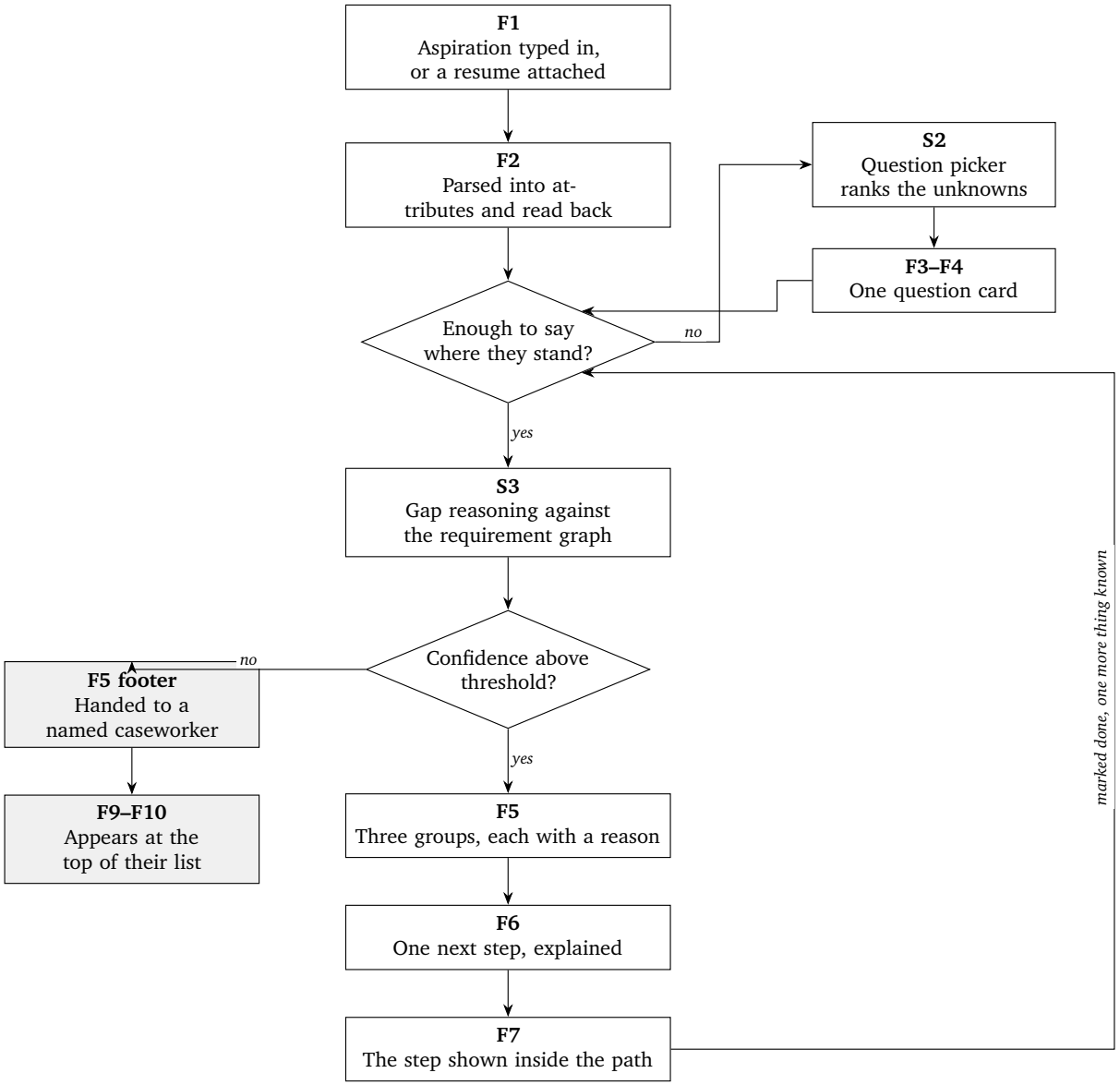


Figure 1: The end-to-end flow. **F***n* are mock-UI frames (Section 4); **S***n* are backend services (Section 5).

The loop matters as much as any single screen. Frame 6 is not the end of the process. Marking the step done sends the user back into the engine with one more attribute known, and the next run gives a different answer.

Worked example

Priya types *"I want to become a data analyst but I don't know where to start."* The parser gets an intent out of that and not much else, and shows it back to her. The picker does not ask for her Aadhaar number or her father's occupation. It asks what kind of work she is picturing, because nothing else can be scored until that is fixed. She taps *working with data and numbers*. The next question follows from that answer: does she have a resume? She taps *no, not yet*.

Two taps in, the system reports that she already has spreadsheet basics, that she is missing a resume and basic SQL, and that three other things can wait. Confidence on the SQL judgement is only partial, so the screen also offers Ravi, a caseworker in Kukatpally who speaks Telugu.

Her next step is not a list of courses. It is: write your first resume, about two hours, bring your

marksheets, and there is a clinic on Saturday if you want help.

4 Functional mock UI

The mock is there to explain the design, not to be built on. We wanted something a Barabari caseworker could look at for two minutes and then tell us what we had got wrong.

There are eleven frames, all mobile-first and sized for a shared budget Android phone. Eight are the student path and three are the caseworker path. They share one set of type, spacing and status colours, and nothing in the product uses red, since none of it is a rejection. Each path also carries its own bottom navigation bar, so the two roles never share a route through the app.

The clickable prototype is on Figma:

`https://www.figma.com/design/9K2MGda8rKSc7ZYwwkkB81/Untitled?node-id=0-1&p=f&t=PYJnIZwnkmY4PRez-0`

4.1 Student path

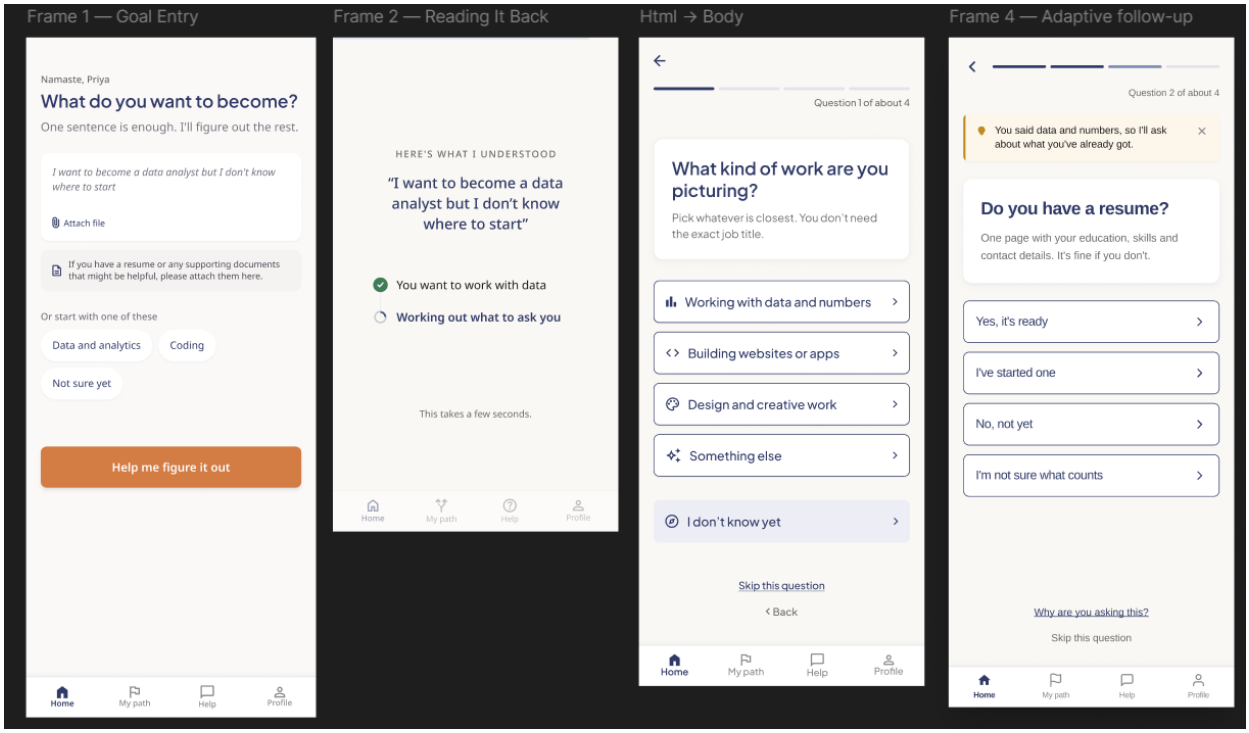


Figure 2: Frames 1–4: goal entry, reading it back, the first question, and the adaptive follow-up. The student bottom bar runs along all four.

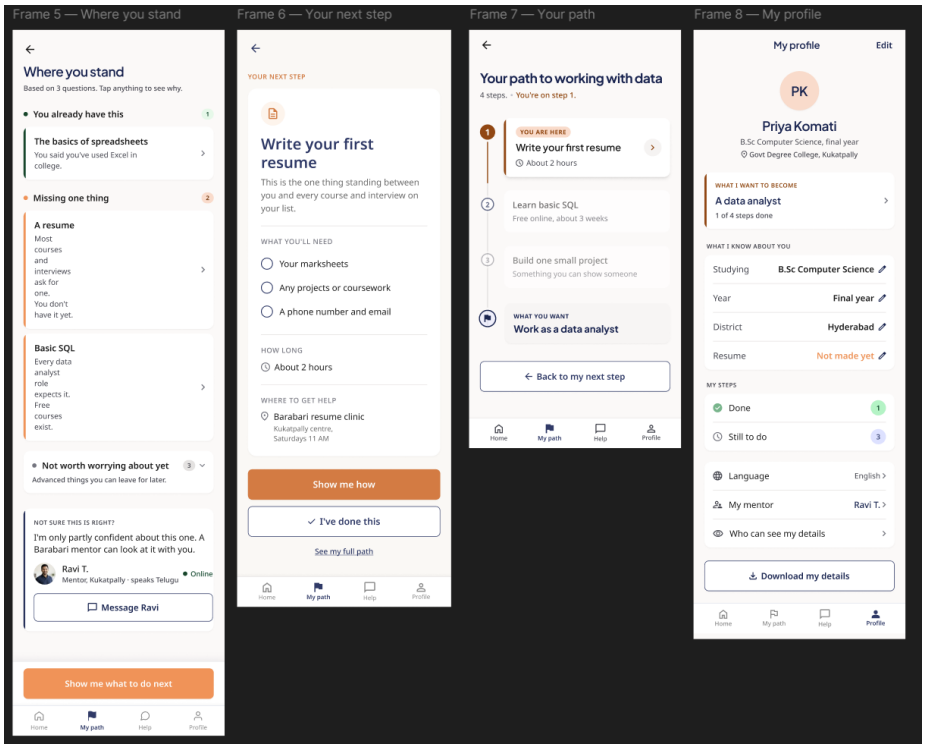


Figure 3: Frames 5–8: where you stand, the single next step, the path it belongs to, and the student's own profile. The caseworker handover sits in the footer of frame 5.

#	Frame	What it needs to show
1	Goal entry	A sentence or a resume is enough to start. No login, no form, no jargon.
2	Reading it back	The sentence was read, not keyword-matched.
3	First question	One question on the screen, and "I don't know yet" treated as a real answer.
4	Adaptive follow-up	The question was picked because of the last answer, and the screen says so.
5	Where you stand	Groups with reasons instead of a ranked list, and a caseworker when confidence drops.
6	Your next step	One action, with what to bring and how long it takes.
7	Your path	The step has a destination, without the distance taking over.
8	My profile	Everything the system holds, visible and editable.

Table 2: Student path, frames 1–8.

4.2 Caseworker path

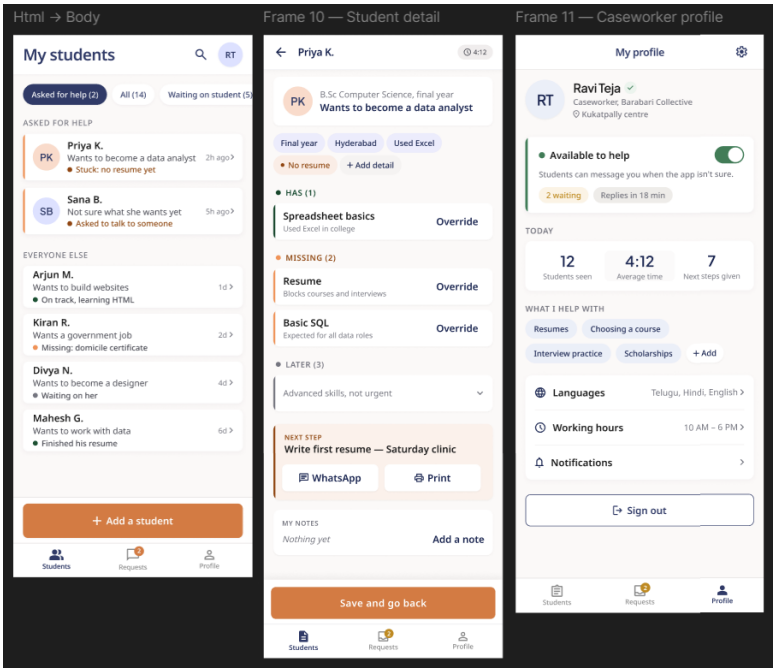


Figure 4: Frames 9–11: the student list with handovers at the top, one student opened with override controls, and the caseworker's availability profile.

#	Frame	What it needs to show
9	Caseworker view	Handovers from frame 5 arrive where the caseworker will see them.
10	Student detail	The same reasoning, denser, with an override on every row.
11	Caseworker profile	The availability toggle that makes the handover in frame 5 real.

Table 3: Caseworker path, frames 9–11.

Two screens carry most of the argument. Frame 6 is the claim that one action beats a complete list. The caseworker card in frame 5 is the claim that a system should hand over rather than invent. Both are assumptions, and both are cheap to be wrong about now.

One inconsistency is deliberate: frames 5 and 6 say *mentor* to the student, while the internal tool says *caseworker*. A student is being offered a person, not routed to a case officer. We would still test this rather than assume it.

5 Core technical component definition

The core is a decision-support and information-gathering engine, built as four services behind one endpoint. The UI, the voice layer and the caseworker dashboard can all be swapped out. These four cannot.

Service	Input	Output	Behaviour
S1 Intake parser	Free text, transcribed speech, or an uploaded resume	Profile JSON with per-field confidence and an explicit `unknown` set	Pulls out what the input contains and marks what it does not. It never fills a slot with a guess. A resume gives education, skills and experience; a sentence gives intent and little else.
S2 Question picker	Partial profile and the live candidate set	One question, its expected information gain, and the reason it was picked	Ranks the unknown attributes by how much an answer would narrow things, returns the top one, and stops once the gain falls below a threshold. The reason string is what frame 4 displays.
S3 Gap reasoner	A complete-enough profile and the requirement graph	Three groups with a reason each, plus the nearest actionable step	Compares what the person has against what the role needs. Where something is missing it walks the graph to the closest node the user could reach in hours or weeks.
S4 Confidence gate	The S3 response with per-item confidence	A pass, or a handover payload	Aggregates confidence across the response. Below threshold, or on an explicit "I don't know", it withholds the answer and emits a handover instead.

Table 4: The four services.

The endpoint `POST /session/next` loops S1 and S2 until S2 reports it has enough, calls S3 once, passes the result through S4, and returns the groups, the reasons, the next step, the remaining unknowns, and either a pass or a handover.

Take out the role-requirement corpus and what is left is general: given a goal, an incomplete profile and a rulebook, ask the fewest questions, return the nearest step, and know when to fetch a human. Scholarship navigation, health-service eligibility and legal-aid intake would each need a different graph but the same engine. Keeping the engine and the corpus separate is the main architectural decision here.

We plan to measure four things on synthetic profiles: the average number of questions before the groups settle (we want under five), the precision of the *already have* group, the share of blocked cases where the system still returns a reachable step, and how often S4 escalates on cases we have labelled ambiguous. The last one is new and we are least sure how to set it.

6 Initial technical approach

Two constraints drove the stack. It has to run on a laptop for a semester demo, and nothing important can depend on a paid API we might lose.

Layer	Choice	Reason
Mock UI	Figma, frames drafted with Stitch	Sprint 1 asks for a navigable mock
Frontend	Next.js, mobile-first	Works in a phone browser with no install
Extraction	Small quantised model via Ollama with schema-constrained output; pdfplumber and Tesseract for resumes	Local, no API key, and we get JSON rather than prose to re-parse
Question picking	Python and NumPy, information gain over the candidate set	Explainable, so we can show why a question was asked
Requirement graph	NetworkX DAG over roles, skills, artefacts and courses	Cheap traversal, and the rules stay readable and hand-correctable
Backend	FastAPI	Thin orchestration over the four services
Storage	PostgreSQL with JSONB	Rule shapes vary between roles
Voice	Bhashini (ASR, MT, TTS)	Deferred until the English text path works

Table 5: Stack, with the reason for each choice.

Data. The requirement graph is curated by hand: for each target role, what it needs and what sits between a student and it. Where a step involves a scholarship we take the rules from myScheme and the OGD Platform, kept to a small corpus rather than national coverage. Test profiles are synthetic. We generate aspirations and resumes and then delete attributes on purpose, so the engine sees the incomplete pictures it is supposed to handle. None of this is real student data, so the numbers we get are a benchmark and not evidence that it works.

Division of labour. The language model does one job, which is turning text into fields. It never decides what a person is missing. Every claim about a gap comes from a check against the graph and carries a confidence value that S4 can act on. This is our answer to the chatbot failure mode: a fluent, confident, wrong answer given to someone who will act on it.

Handover. Escalation runs over WhatsApp rather than in-app chat. Students already have it, caseworkers already work in it, and building a messaging surface is exactly the kind of production plumbing this project is scoped to avoid.

7 Scope, assumptions and next steps

Out of scope

No login or user management, no admin dashboard, no integration plumbing. No job listings or applications; the product explains and directs, it does not place anyone. No national scheme ingestion, only a curated corpus. No built frontend this sprint.

Assumptions we want Barabari to correct

1. **That the blocker is information.** We have assumed a student who knew the right step would take it. If the real obstacle is household permission, a fee, or a belief that an opportunity is not meant for them, better information changes nothing. We want to ask caseworkers what actually stops a student halfway.
2. **That requirements can be codified cheaply.** Whether what a role needs converts into a checkable graph without months of manual work. We will build three roles and report the real cost per role.
3. **That a question sequence is tolerable.** Three or four questions should beat a form, but sequential can also feel slow. Worth testing before we commit.
4. **That one step beats the whole list.** A claim about motivation rather than technology, and caseworkers have watched it play out.
5. **That the handover threshold can be set.** Escalate too readily and we have built a queue instead of relief from one. Too rarely and the system is wrong where it matters. We want caseworkers to tell us which error they would rather absorb.
6. **That synthetic profiles are messy enough.** Real students have conflicting records and colloquial phrasing our generator will not produce. We would like to check against anonymised case notes if partners can share them.

Sprint 2

- Build the requirement graph for three roles.
- Implement S3 first. It is the hardest piece and the other three exist to feed it.
- Write the synthetic profile generator with deliberate attribute dropout.
- Implement S4 as a simple threshold and instrument it so it can be tuned.
- Get a baseline for questions-to-settle and gap-resolution rate.
- Take frame 6 and the frame 5 caseworker card to Barabari and record the disagreements.

Checkpoint

We are building a next-step engine that turns a stated aspiration or a resume into one concrete action, names what is missing, and hands over to a caseworker when it is not confident. The technology is four services: an intake parser, an information-gain question picker, a deterministic gap reasoner over a requirement graph, and a confidence gate. The reason is that every existing route answers a different question, and none of them tells this student what is missing from their own picture.

Sources

- myScheme — central and state scheme rules.
- OGD Platform India — open government datasets.
- Team DSIders Sprint 0 functional scope document and notes on Barabari Collective.
- DSI project brief 01B, *Finding the Next Best Step*.